

(Q) Logic

→ We will first find the median of values using MOM

→ In the code update during rearrangement of values, rearrange their corresponding weights as well

value	1	3	3	2	4
weight	6	7	8	9	10

After rearranging	Value	1	2	3	5	4
weight	→	6	9	8	7	10

Just need ensure this as well

→ The calculate the sum of weights which are left to median

→ If they are less than $\left[\frac{\text{total sum of weights}}{2} \right]$ then proceed

→ find median sum of weights to right of median

→ If ~~this~~ this is less than $\left[\frac{\text{total sum}}{2} \right]$ median

found is the Ans

→ else find another median in right subpart of the array and again follow same procedure

→ else find ^{another} median in right subpart of the array then repeat same algo

→ Do this until we find our Ans

* for ensuring value & corresponding weights go altogether during rearrangement in MOM use pair

→ first element = value

Second element = weight

pseudo code as a global variable / anything

total-sum weight[]

find (start, end, arr [],)

mid = find-median (start, end, arr [])

← using MDH

for (i = start → mid)

left-sum += weight[i]

(therefore all value to left of median in arr will be less than median)

for (i = mid → end)

right-sum += weight[i]

all values to the right will be greater than median

if (left-sum < $\frac{\text{total-sum}}{2}$)

{ if (right-sum < $\frac{\text{total-sum}}{2}$)

{

return mid

}

else

{

return find (mid, end, arr, weight)

}

}

else

{

return find (start, mid, arr, weight)

* In find-median it ~~doesn't actually~~ partially sort
↳ add the code s.t

rearranging the array wrt median ensuring that values less than median are to left & values greater than median are to right.

Proof of Algorithm to be $O(n)$

Say complexity of algorithm on n element $= T(n)$

$$T(n) = \underbrace{cn}_{\downarrow} + T\left(\frac{n}{2}\right)$$

finding median takes $O(n)$
 complexity through MoM concept

↳ non rearranging values
 at max n operations

↳ Also finding left & right sum takes n operations

$$\Rightarrow T\left(\frac{n}{2}\right) = c\frac{n}{2} + T\left(\frac{n}{4}\right)$$

$$T\left(\frac{n}{4}\right) = c\frac{n}{4} + T\left(\frac{n}{8}\right)$$

$$+ T(n) = cn + \left(\frac{n}{2}\right) + \left(\frac{n}{4}\right) + \dots + T(1)$$

$$\boxed{T(n) \leq 2cn}$$

$$\Rightarrow \boxed{T(n) = O(n)}$$

Hence the given algo works in $O(n)$

complexity

for MoM (Assume $K(n)$)

↳ Dividing array into subarray of elements and sorting them

$O(n)$

$O(1)$ } Since S elements are sorted in constant time.

↳ Recursive function call for finding median of medians

size = $\left(\frac{n}{3}\right)$

$$\hookrightarrow K\left(\frac{n}{3}\right)$$

↳ Partition Done in $O(n)$

↳ Then ~~find median~~ recurse as per pivot

$$\hookrightarrow K\left(\frac{7n}{10}\right)$$

$$1 \quad \textcircled{K(n)} \quad K(n) = K\left(\frac{n}{3}\right)$$

Since atleast $\frac{3n}{10}$ will be removed

$$\Rightarrow K(n) = K\left(\frac{n}{3}\right) + K\left(\frac{7n}{10}\right) + O(n)$$

$$K\left(\frac{7n}{10}\right) \leq K\left(\frac{7}{10} \cdot \frac{7n}{10}\right) + \dots + \frac{7n}{10}$$

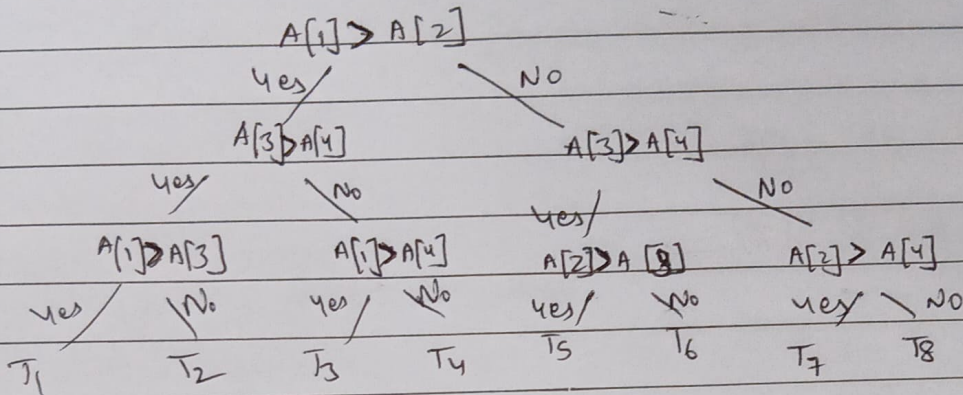
$$\Rightarrow K(n) \leq K\left(\frac{7n}{10}\right) + \textcircled{O(n)} + cn$$

O ;

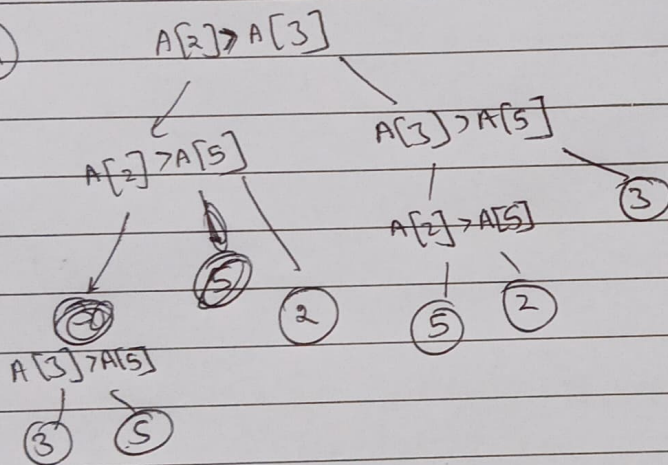
$$\Rightarrow K(n) \leq K(1) + \left(\frac{7n}{10} + \frac{7}{10} \cdot \frac{7n}{10} + \dots\right) \Rightarrow \boxed{K(n) \leq \frac{c \cdot 7n}{3}} \Rightarrow \boxed{K(n) = O(n)}$$

(2)

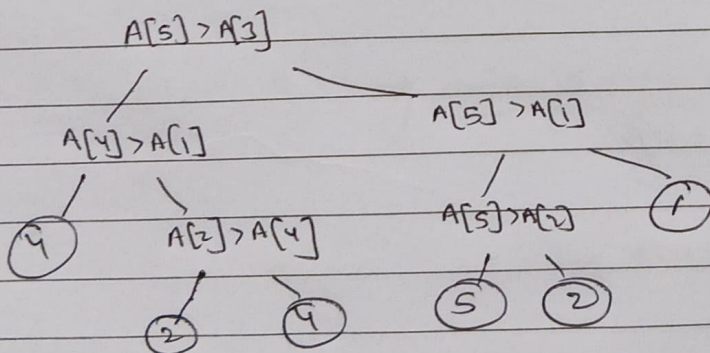
To determine median following is the required decision tree



(T1)

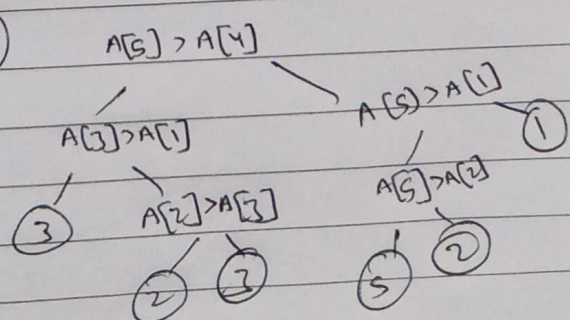
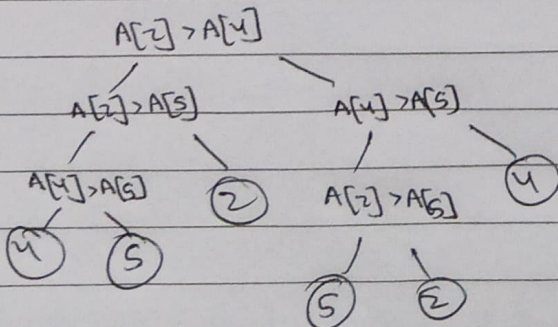


(T2)

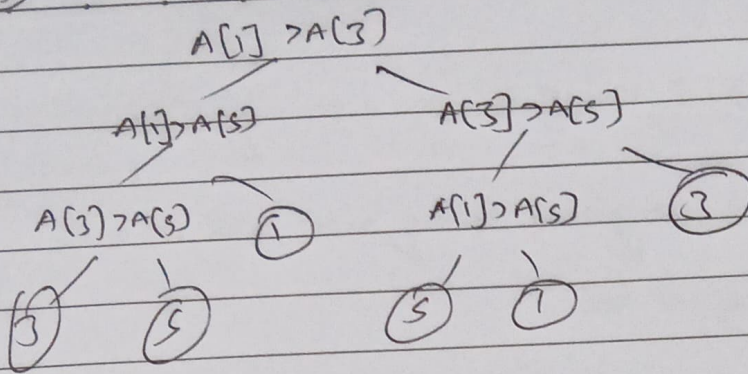


(T4)

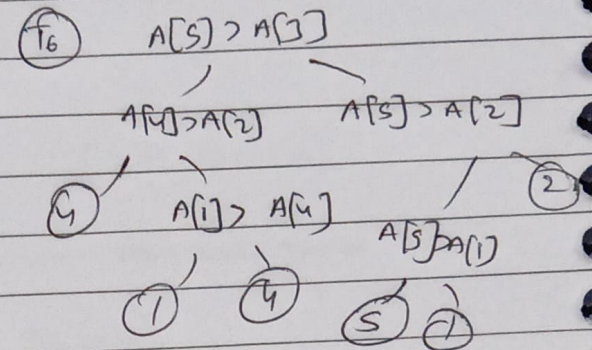
(T)



⑤

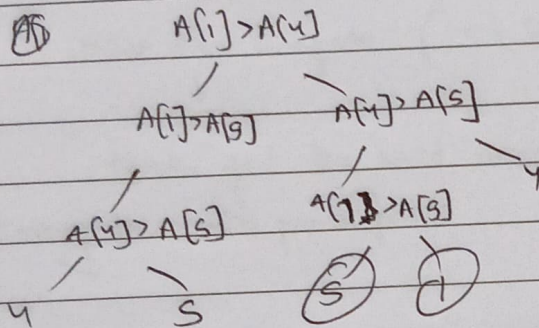


⑥

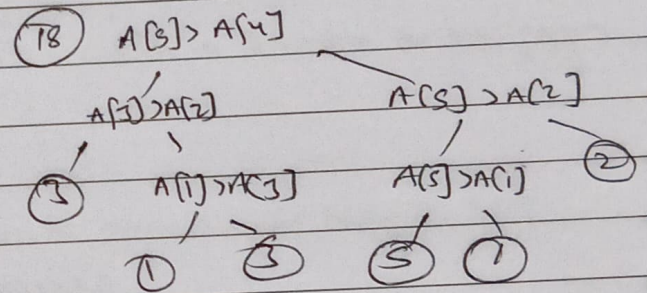


⑦

⑧



⑧



Explanation of correctness

Since all elements are distinct each comparison gives a definitive order b/w two elements.

The median is the third element by systematically comparing elements we eliminate the smallest and largest candidates, ensuring the median is correctly identified.

Also, each path is constructed such that no more than six comparisons are needed to reach a leaf where the median is identified.

Ass-4

Q-3

a) Divide 100 programmers into 10 groups of 10 people each

In 10 sessions these groups participants compete (10 sessions)

↓
Taking the winner from each group

Now we have, 10 players (candidates for being the best)

↓
Make them compete (+1 Session)

↓
Now get the best among the best, which is therefore the best programmer among the 100 participants

b) Since in one session at max $\rightarrow$ 9 candidates can be removed for being the best therefore we need more than 10 sessions

Since in 10 Sessions at max 90 candidates can be removed

Therefore it is impossible to find best programmer in only 10 sessions.

c) Using the same strategy as above,

In 10 sessions find 10 best players among their groups

Now in the 11th sessions

we get top 4 participants

↓

a, b, c, d

↓
1st 2nd 3rd 4th

↳ But also keep note of top 1 players from each group including these players

Take top 4 players from the group of (a) (including a) [though, only 2nd, 3rd, 4th would also work since a is already best]

Take top 3 players from the group of (b) (including b)
Take top 2 " " " " " " " "
Take top 1 " " " " " " " "

Now play a session with top 4 from group of (a)
 top 3 from group of (b)
 top 2 from group of (c)
 and (d) } total of 10 participants.

This will give you the top 4 players &

why we took only some specific no of players?

↳ Since top 4 players from group of (a) are legitimate candidate for being top 4 among all the players

↳ But for group of b since a is better than b & therefore better than all the players of group of b

↓
 Hence only 3 candidates are remaining
 Hence 3

↳ Similarly for c, a & b are better than it therefore only 2 candidates are remaining

Therefore in 12 sessions we can

easily find top 4 players/leaders among the 100

Ans

↳ 10 sessions for 10 groups
 ↳ 1 session for winners of 10 groups
 ↳ 1 session in the way mentioned above